

Lab Manual 5: Interfacing Display Peripheral with Microcontroller

Goals

- Understand memory-mapped I/O with the ARM architecture.
 - Learn how to interface display and input peripherals using ARM Assembly.
 - Perform read/write operations on GPIO devices.
 - Explore VGA graphics, serial communication, and seven-segment display handling.
-

1. Interfacing LEDs

Address: `0xFF200000` (Base address for LEDs)

Access Method:

- The LEDs are controlled using word (32-bit) write operations.
- This means writing a full word to address `0xFF200000` controls all 32 LEDs at once.
- Each LED corresponds to one of 32 bits in the word (LED0 to LED31).
- Writing a `1` turns ON the respective LED; a `0` turns it OFF.
- **The GPIO device may not support byte, half-word, or misaligned stores. Use aligned word-sized stores.** (<https://cpulator.01xz.net/wiki/Msg/DevGpio>)

Addressing and Bit Mapping:

Bit:	31		2	1	0
LED:	LED31		LED2	LED1	LED0

Example 1 – Turn ON LED0:

```
.global _start
_start:
    LDR R0, =0xFF200000    @ Load base address of LED register into R0
    MOV R1, #0x1           @ Prepare data: turn on LED bit 0
    STR R1, [R0]           @ Write value to LED register

end:
    B end                  @ Infinite loop to stop the program
```

Example 2 – Turn ON LEDs (LED9, LED5, LED1):

```
.global _start
_start:
    LDR R0, =0xFF200000    @ Load base address of LED register
    LDR R1, =0x122         @ Load value 0x122 (binary 100100010) into R1
    STR R1, [R0]           @ Store value to LED register

end:
    B end                  @ Infinite loop to end program
```

Example 3 – Blink LED0:

```
.equ LED_BASE, 0xFF200000    @ Base address of the LED register
.equ DELAY_COUNT, 500000    @ Loop count for delay (adjust to change
delay length)

.global _start               @ Program entry point
_start:
    LDR R0, =LED_BASE        @ Load LED base address into R0

blink:
    MOV R1, #0x1             @ Turn on LED bit 0
    STR R1, [R0]             @ Write to LED register

    BL delay                 @ Call delay routine

    MOV R1, #0x0             @ Turn off all LEDs
    STR R1, [R0]             @ Write to LED register

    BL delay                 @ Call delay again

    B blink                  @ Repeat blinking loop

@ -----
@ Delay routine: busy loop delay
@ -----
delay:
    PUSH {R0, R1, R2}        @ Save registers
```

```

MOV R1, #0           @ Initialize counter
LDR R2, =DELAY_COUNT @ Load delay loop limit

loop:
ADD R1, R1, #1       @ Increment counter
CMP R1, R2           @ Compare with limit
BLE loop             @ Continue loop while counter <= limit

POP {R0, R1, R2}     @ Restore registers
BX LR                @ Return from subroutine

```

3. Interfacing Switches

Address: `0xFF200040` (Base address for switches)

Access Method:

- The switches are represented as a 32-bit input.
- Reading a full word (`LDR`) from address `0xFF200040` returns the current state of all 32 switches (SW0 to SW31).
- Each bit in the word corresponds to a switch:
 - Bit 0 → SW0
 - Bit 1 → SW1
 - ...
 - Bit 31 → SW31
- A bit value of `1` means the corresponding switch is **ON**, and `0` means it is **OFF**.

Byte-Level Access (Optional):

- `0xFF200040`: Reads switches SW0–SW7
- `0xFF200041`: Reads switches SW8–SW15
- `0xFF200042`: Reads switches SW16–SW23
- `0xFF200043`: Reads switches SW24–SW31

Addressing and Bit Mapping:

Bit:	31		2	1	0
Switches:	SW31		SW2	SW1	SW0

Example – Read switch and write to LED:

```

.equ LED_BASE,    0xFF200000 @ Base address for LEDs
.equ SWITCH_BASE, 0xFF200040 @ Base address for Switches

```

```

.global _start
_start:

loop:
    LDR R0, =SWITCH_BASE           @ Load switch base address
    LDR R1, [R0]                   @ Read all switch states
    LDR R2, =LED_BASE              @ Load LED base address
    STR R1, [R2]                   @ Write switch states to LEDs
    B loop                         @ Repeat indefinitely

```

4. Interfacing Push Buttons

Address: `0xFF200050` (Base address for push buttons)

Access Method:

- The push buttons are represented as a 32-bit input.
- Reading a full word (`LDR`) from address `0xFF200050` returns the current state of all 32 push buttons (BTN0 to BTN31).
- Each bit in the word corresponds to a switch:
 - Bit 0 → BTN0
 - Bit 1 → BTN1
 - ...
 - Bit 31 → BTN31
- A bit value of `1` means the corresponding switch is **ON**, and `0` means it is **OFF**.

Byte-Level Access (Optional):

- `0xFF200050`: Reads push buttons BTN0–BTN7
- `0xFF200051`: Reads push buttons BTN8–BTN15
- `0xFF200052`: Reads push buttons BTN16–BTN23
- `0xFF200053`: Reads push buttons BTN24–BTN31

Addressing and Bit Mapping:

Bit:	31		2	1	0
Switches:	BTN31		BTN2	BTN1	BTN0

Example – Toggle LED0 on BTN0 Press:

```

.equ LED_BASE,    0xFF200000
.equ BUTTON_BASE, 0xFF200050

```

```

.global _start
_start:

    MOV R3, #0          @ R3 holds LED state (0 = OFF, 1 = ON)
    MOV R4, #0          @ R4 holds previous button state (0 = released, 1 =
pressed)

loop:
    LDR R0, =BUTTON_BASE
    LDRB R1, [R0]        @ Read button state
    AND R1, R1, #0x1     @ Mask KEY0 bit

    CMP R1, R4           @ Compare current and previous button state
    BEQ no_change        @ If same, no change, skip toggle

    CMP R1, #1           @ Check if button is pressed (rising edge)
    BNE no_change        @ If not pressed, no toggle

    @ Toggle LED state
    EOR R3, R3, #1       @ XOR R3 with 1 → toggle bit

    LDR R2, =LED_BASE
    STR R3, [R2]         @ Output LED state to LED0

no_change:
    MOV R4, R1           @ Save current button state as previous
    B loop               @ Repeat forever

```

5. Seven-Segment Display Interface

Addresses:

- 0xFF200020 controls HEX0 to HEX3
- 0xFF200030 controls HEX4 to HEX7

Access Method:

- The displays are controlled via **32-bit word writes**.
- Writing a full 32-bit word to 0xFF200020 sets the values for HEX0–HEX3 simultaneously.

- Writing a full 32-bit word to **0xFF200030** sets the values for HEX4–HEX7 simultaneously.
- The GPIO device may not support byte, half-word, or misaligned stores. Use aligned word-sized stores. (<https://cpulator.01xz.net/wiki/Msg/DevGpio>)

Bit and Address Mapping:

For address **0xFF200020** (HEX0–HEX3):

Byte Address	Bits	HEX Display
0xFF200020	Bits [7:0]	HEX0
0xFF200021	Bits [15:8]	HEX1
0xFF200022	Bits [23:16]	HEX2
0xFF200023	Bits [31:24]	HEX3

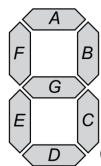
For address **0xFF200030** (HEX4–HEX7):

Byte Address	Bits	HEX Display
0xFF200030	Bits [7:0]	HEX4
0xFF200031	Bits [15:8]	HEX5
0xFF200032	Bits [23:16]	HEX6
0xFF200033	Bits [31:24]	HEX7

Segment Bit Mapping:

- Each HEX display is 8 bits wide, but only the lower 7 bits (bits [6:0]) are used to control the actual segments (G–A). The last bit (bit 7) is ignored and has no effect on the display in CPulator.
- A bit set to 1 turns ON the respective segment of the HEX display.

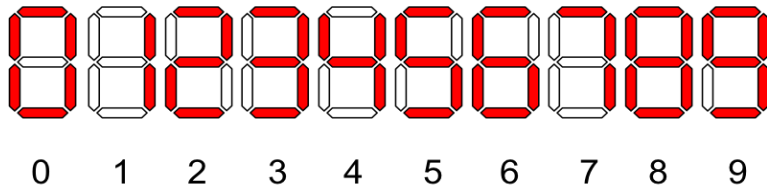
Common Digit Encoding (for HEX Displays):



Digit	Encoding (Hex)	Binary (bits [6:0])	Segments ON
-------	----------------	---------------------	-------------

0	0x3F	0b01111111	F, E, D, C, B, A
1	0x06	0b00001110	C, B
2	0x5B	0b10110111	G, E, D, B, A
3	...	...	...

Note: You can also expand the table to include other digits and characters



0 1 2 3 4 5 6 7 8 9

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Upper	8		0		6	6	6	8	1	8		8	00		0	8			5		0		00			2
Lower	8	6	2	8	2		9	8	6	8		8		8	6		9	2		8	0				9	

Example 1 – Display 0 on HEX0:

```
.equ HEX0_BASE, 0xFF200020 @ Base address for HEX0 (lowest byte at this
address)

.global _start
_start:
    LDR R0, =HEX0_BASE @ Load address of HEX0 display
    MOV R1, #0x3F @ Encoding for digit 0 (segments A, B, C, D, E, F ON)
    STR R1, [R0] @ Write 1 byte to HEX0 (only lowest 7 bits used)

end:
    B end @ Infinite loop to keep the program running
```

Example 2 – Display HELLO:

```
.equ HEX03_BASE, 0xFF200020 @ Base address for HEX0-HEX3
.equ HEX4_BASE, 0xFF200030 @ Base address for HEX4
```

```

.global _start
_start:
    LDR R0, =HEX0_BASE          @ Load address for HEX0-HEX3

    @ -----
    @ Combined binary literal: 0b01111001_00111000_00111000_00111111
    @ Each byte corresponds to one HEX digit (right to left):
    @
    @  HEX0 = 0x76 = 0b00111111 ('O')
    @  HEX1 = 0x79 = 0b00111000 ('L')
    @  HEX2 = 0x38 = 0b00111000 ('L')
    @  HEX3 = 0x38 = 0b01111001 ('E')
    @
    @ Little-endian memory layout:
    @  [HEX3][HEX2][HEX1][HEX0] ← MSB ..... LSB
    @ -----

    LDR R1, =0b01111001001110000011100000111111 @ Encoded "ELLO"
    STR R1, [R0]                                   @ Store full word to
HEX3-HEX0

    @ Now write 'H' to HEX4
    LDR R2, =HEX4_BASE
    MOV R3, #0b01110110                            @ Encoding for 'H'
    (segments A-F ON)
    STRB R3, [R2]                                   @ Store byte to HEX4

loop:
    B loop                                           @ Infinite loop to hold
the display

```

Example 3 – "Loading" animation inside a single HEX (A → B → C → D → E → F → A → ...):

```

.equ HEX_BASE, 0xFF200020    @ Base address for HEX0 display

.global _start
_start:
    LDR R0, =HEX_BASE        @ Load base address of HEX0 display into R0
    MOV R1, #0x1             @ Start with segment A (bit 0, corresponds
to segment G if following original mapping)

```



```

loop:
    MOV R2, R1
    STR R2, [R0]                @ Write the 32-bit word to HEX0 (only the
lowest byte matters)

    BL delay                    @ Call delay routine

    @ Move to next segment by shifting left
    CMP R1, #0x20                @ If current segment is bit 5 (segment F)
    BEQ reset_to_a              @ If yes, reset to segment A (bit 0)

    LSL R1, R1, #1                @ Otherwise, shift left to next segment bit
    B loop                        @ Repeat the loop

reset_to_a:
    MOV R1, #0x1                @ Reset segment bit to 0x01 (segment A)
    B loop                        @ Repeat the loop

delay:
    LDR R3, =0x80000            @ Load delay count into R3 (adjust for
speed)
wait:
    SUBS R3, R3, #1              @ Decrement delay counter
    BNE wait                    @ Loop until zero
    BX LR                       @ Return from delay

```

Example 4 – Scrolling Text animation ('HELLO') on HEX0–HEX3:

```

.equ HEX03_BASE, 0xFF200020    @ Base address for controlling HEX0-HEX3
displays

.global _start
_start:
    LDR R6, =letters            @ R6 points to the start of the encoded
"HELLO" array
    LDR R7, =HEX03_BASE        @ R7 holds the base address of the
7-segment display controller

    @ -----
    @ Construct initial 32-bit value (R0) representing 4 letters: H, E, L,
L

```

```

@ Letters packed in order from left to right:
@  HEX3 ← HEX2 ← HEX1 ← HEX0
@  -----

LDR R0, [R6]                @ Load 1st letter 'H' into R0

LDR R1, [R6, #4]!           @ Load 2nd letter 'E' and increment R6
LSL R0, R0, #8              @ Shift 'H' left 8 bits
ORR R0, R0, R1              @ R0 = 'H'E'

LDR R1, [R6, #4]!           @ Load 3rd letter 'L', R6 += 4
LSL R0, R0, #8              @ Shift left 8 bits
ORR R0, R0, R1              @ R0 = 'H'E'L'

LDR R1, [R6, #4]!           @ Load 4th letter 'L', R6 += 4
LSL R0, R0, #8              @ Shift left 8 bits
ORR R0, R0, R1              @ R0 = 'H'E'L'L' → initial display

LDR R1, [R6, #4]!           @ Load 5th letter 'O' into R1 (used during
rotation)

@ -----
@ Main loop: Rotate letters across HEX0-HEX3
@ -----
loop:
    STR R0, [R7]             @ Output current 32-bit value to display
    (HEX0-HEX3)
    BL delay                 @ Software delay for animation pacing

    @ -----
    @ Rotation logic:
    @  Shift letters left by 1 byte (discard HEX3),
    @  Append previous HEX3 letter to HEX0 (rightmost).
    @  Example: H-E-L-L → E-L-L-O → L-L-O-H → ...
    @ -----

    LSR R2, R0, #24          @ Extract HEX3 (most significant byte) into
R2
    LSL R0, R0, #8           @ Shift R0 left by 1 byte
    ORR R0, R0, R1           @ Append last HEX3 (stored in R1) to HEX0
    MOV R1, R2              @ Update R1 with rotated-out letter for
next round

```

```

B loop                                @ Repeat endlessly

@ -----
@ Delay routine: simple loop-based delay
@ -----
delay:
    MOV R2, #0x400000                @ Delay counter (adjust to control speed)
delay_loop:
    SUBS R2, R2, #1                  @ Decrement counter
    BNE delay_loop                  @ Loop until counter reaches 0
    BX LR                          @ Return from delay

@ -----
@ Data section: 7-segment encoding for "HELLO"
@ Each word = 8-bit value to control one HEX digit (only bits [6:0] used)
@ -----
.data
letters:
    .word 0x76    @ H → Segments A, B, C, E, F, G
    .word 0x79    @ E → Segments A, D, E, F, G
    .word 0x38    @ L → Segments D, E, F
    .word 0x38    @ L → Same as above
    .word 0x3F    @ O → Segments A, B, C, D, E, F

```

6. JTAG UART

Address: 0xFF201000 (used for both data and control)

Access Method:

- Bits [7:0] are used for data transmission and reception.
- Reading:
 - Bit 15 is the read-ready flag. 1 = Data is available to read.
 - Always check bit 15 before reading to ensure valid data.
- Writing:
 - On CPulator, data can be written directly to bits [7:0] without checking a write-ready flag.
 - In general, UARTs, a separate write-ready flag may be required, but CPulator simplifies this.
- Note: This memory-mapped I/O register serves both control and data functions through the same address.

Example 1 – Write a character 'A' on JTAG UART:

```
.equ UART, 0xFF201000      @ JTAG UART data/control register

.global _start
_start:
    LDR R0, =UART           @ Load UART address into R0
    MOV R1, #0x41           @ ASCII for 'A'
    STRB R1, [R0]           @ Write 'A' to UART (lower 8 bits only)

end:
    B end                   @ Infinite loop to stop program
```

Example 2 – Write a string 'Hello World!' on JTAG UART:

```
.equ UART, 0xFF201000      @ UART base address

.global _start
_start:
    LDR R0, =UART           @ Load UART address into R0
    LDR R1, =msg            @ Load address of the string into R1

print_loop:
    LDRB R2, [R1], #1       @ Load one byte from string and increment
                             pointer
    CMP R2, #0              @ Check if byte is null terminator (end of
                             string)
    BEQ done                @ If null, finish printing
    STRB R2, [R0]           @ Write byte to UART (send character)
    B print_loop            @ Repeat for next character

done:
    B done                  @ Infinite loop when done

.data
msg:
    .asciz "Hello World!"   @ Null-terminated string to print
```

Example 3 – Read a Number (One Character) from JTAG UART:

```
.equ UART, 0xFF201000      @ JTAG UART address (data and control)
```

```

.global _start
_start:
    LDR R0, =UART          @ UART base address

wait_for_data:
    LDR R1, [R0]            @ Read 32-bit UART register (data + status)
    TST R1, #(1 << 15)      @ Test bit 15 (read-ready flag)
    BEQ wait_for_data       @ If not ready, wait here

    AND R2, R1, #0xFF       @ Extract received byte (bits [7:0])
    @ Now R2 holds the received character

    B done                  @ End program (or continue processing)

done:
    B done                  @ Infinite loop

```

Example 4 – Reading and Accumulating Numeric Input from JTAG UART with Enter Key Detection:

```

.equ UART, 0xFF201000

.global _start
_start:
    LDR R0, =UART          @ UART base address
    MOV R4, #0              @ R4 will hold the accumulated number

read_loop:
    @ Wait for data available
    LDR R1, [R0]
    TST R1, #(1 << 15)
    BEQ read_loop

    AND R2, R1, #0xFF       @ Extract received byte

    CMP R2, #10              @ Check for Enter key (ASCII 10)
    BEQ done                 @ Exit if Enter pressed

    SUBS R2, R2, #'0'        @ Convert ASCII digit to number
    BLO read_loop            @ Ignore if less than '0'
    CMP R2, #9
    BHI read_loop            @ Ignore if greater than '9'

```

```

MOV R3, #10           @ Load immediate 10 into R3 for multiplication
MUL R4, R4, R3         @ Multiply accumulated number in R4 by 10
ADD R4, R4, R2         @ Add the new digit (in R2) to R4

```

```

B read_loop

```

```

done:

```

```

@ R4 contains the complete number
B done           @ Infinite loop to halt

```

7. VGA

VGA Pixel Buffer:

- **Base Address:** `0xC8000000`
- **Pixel Format:** 16-bit RGB565 (2 bytes per pixel)
- **Width:** 1024 pixels per row
- **Address Calculation:** $\text{Address} = \text{BASE} + (y * 1024 + x) * 2$
- **Access:** Use `STRH` to write pixel data
- **Control Register:**
 - **Address:** `0xFF203020`
 - **Purpose:** Manage VGA display modes, check status, control sync, and refresh
 - **Usage:** Optional for simple pixel writes; used for advanced VGA configuration

Character Buffer:

- **Base Address:** (Platform dependent, e.g., `0xC9000000`)
- **Purpose:** Stores ASCII or encoded characters for text display on screen
- **Access:** Write characters sequentially to display text
- **Control Register:**
 - **Address:** (Platform dependent, e.g., `0xFF203000`)
 - **Purpose:** Control cursor position, blinking, enable/disable text display, and screen refresh
 - **Usage:** Write commands or flags to manage text output

Example 1 – Fill Entire VGA Screen with Red Color (RGB565) on CPulator:

```

.equ VGA_BASE, 0xC8000000    @ Base address of VGA pixel buffer
(framebuffer)

```

```

.equ WIDTH, 1024                @ Width of the screen in pixels
.equ HEIGHT, 768                @ Height of the screen in pixels (adjust as
needed)

.global _start
_start:
    MOV R4, #0                  @ Initialize row counter y = 0

fill_loop_y:
    CMP R4, #HEIGHT             @ Compare y with screen height
    BGE done                    @ If y >= HEIGHT, all rows done, jump to done

    MOV R5, #0                  @ Initialize column counter x = 0

fill_loop_x:
    CMP R5, #WIDTH              @ Compare x with screen width
    BGE next_row                @ If x >= WIDTH, go to next row

    @ Calculate pixel address in bytes:
    @ Address = VGA_BASE + ((y * WIDTH) + x) * 2
    MOV R0, R4                  @ Move y to R0
    LSL R0, R0, #10             @ Multiply y by 1024 (width)
    ADD R0, R0, R5              @ Add x to get pixel offset
    LSL R0, R0, #1              @ Multiply by 2 because each pixel is 2 bytes
                                (16-bit)

    LDR R1, =VGA_BASE           @ Load base address of VGA framebuffer
    ADD R1, R1, R0              @ Calculate final pixel address

    MOV R2, #0xF800             @ Set pixel color to red in RGB565 format
                                (red bits set)
    STRH R2, [R1]               @ Store halfword (16 bits) to pixel address

    ADD R5, R5, #1              @ Increment x (column)
    B fill_loop_x               @ Repeat for next pixel in the row

next_row:
    ADD R4, R4, #1              @ Increment y (row)
    B fill_loop_y               @ Repeat for next row

done:
    B done                      @ Infinite loop to halt program after filling
screen

```

Example 2 – Write "Hello World!" to the VGA character buffer on CPulator:

```
.equ CHAR_BASE, 0xC9000000    @ Character buffer base address

.data
string:
    .asciz "Hello World!"    @ Null-terminated string to display

.text
.global _start
_start:
    LDR R0, =string          @ Load address of the string into R0
    LDR R1, =CHAR_BASE       @ Load base address of VGA character buffer
into R1
    MOV R2, #0               @ Initialize index/counter (offset)

write_loop:
    LDRB R3, [R0, R2]        @ Load byte (character) at offset R2
    CMP R3, #0               @ Check for null terminator
    BEQ done                 @ If zero, end loop

    STRB R3, [R1, R2]        @ Store character byte to VGA buffer at
offset R2

    ADD R2, R2, #1           @ Increment offset/index
    B write_loop             @ Repeat loop

done:
    B done                   @ Infinite loop to keep program running
```